

Pythonist のための Haskell 入門

－ 手続き・オブジェクト指向から型付き関数型の考え方へ －

Hiroki Funashima

Kindai University Technical College

Sunday, April 26, 2026

1 この文書の狙い

この文書は、Python に慣れた人が Haskell を読む・書くための導入ノートである。Python と Haskell は、どちらも高水準なプログラミング言語であり、関数を第一級の値として扱える。しかし、実際に書いてみると、その発想はかなり異なる。

Python では、次のような書き方が自然である。

- 変数に値を代入する。
- リストや辞書を更新する。
- for 文で逐次的に処理する。
- 例外を投げて失敗を扱う。
- クラスによってデータと振る舞いをまとめる。

一方、Haskell では次のような発想が中心になる。

- 値は基本的に不変である。
- 関数は入力から出力を作る写像として考える。
- 型によってデータの構造を明示する。
- 失敗や副作用を型で表す。
- パターンマッチによって場合分けする。
- IO は通常の純粋な計算と分離して扱う。

したがって、Haskell を Python の別構文として理解しようとする、かえって分かりにくい。本ノートでは、Pythonist がすでに持っている知識を足場にしつつ、Haskell の考え方へ橋渡しする。

2 Python と Haskell の大まかな違い

最初に、Python と Haskell の違いを大まかに整理しておく。

観点	Python	Haskell
評価モデル	主に逐次実行	式の評価、遅延評価
変数	再代入が普通	値は基本的に不変
型	動的型付け	静的型付け・型推論
関数	第一級オブジェクト	第一級オブジェクト
副作用	通常の処理に混在	IO 型で明示
エラー処理	例外が中心	Maybe, Either など多用
データ構造	dict, list, class が中心	代数的データ型が中心
抽象化	クラス・プロトコル・関数	型・型クラス・高階関数

Python は、手続き型、オブジェクト指向、関数型的な書き方を柔軟に混ぜられる言語である。Haskell は、純粋関数型を中心に設計された言語であり、プログラムの構造を型と関数によって明示する傾向が強い。

この違いは、単に構文の違いではない。プログラムをどう理解し、どう分割し、どう安全性を確保するかという思想の違いである。

3 変数ではなく値を見る

Python では、変数への代入と再代入が自然である。

```
x = 10
x = x + 1
print(x)
```

このコードでは、`x` という名前が最初は `10` を指し、その後で `11` を指すようになる。

一方、Haskell では、次のように考える。

```
x :: Int
x = 10
```

ここで `x` は、`10` という値に束縛 (binding) されている。同じスコープ内で `x` に別の値を再代入することはできない。

したがって、Python の

```
x = x + 1
```

に対応する Haskell の発想は、同じ `x` を更新することではなく、新しい値を作ることである。

```
x :: Int
x = 10

y :: Int
y = x + 1
```

これは不便に見えるかもしれない。しかし、研究用コードや数理的なコードでは、値が途中で変わらないことは大きな利点になる。なぜなら、ある名前が何を表すかが安定し、推論しやすくなるからである。

4 関数の基本

Python の関数は次のように書く。

```
def square(x):  
    return x * x
```

Haskell では次のように書く。

```
square :: Int -> Int  
square x = x * x
```

一行目は型宣言である。

```
square :: Int -> Int
```

これは、square が Int を受け取り、Int を返す関数であることを表す。

二行目は定義である。

```
square x = x * x
```

Haskell では、多くの場合、return は書かない。関数本体の式そのものが返り値である。

Python の

```
def add(x, y):  
    return x + y
```

に対応する Haskell は、

```
add :: Int -> Int -> Int  
add x y = x + y
```

である。

ここで型

```
Int -> Int -> Int
```

は、Int を二つ受け取って Int を返す関数、と読めばよい。

厳密には、Haskell の関数は一引数関数として扱われる。つまり、

```
add :: Int -> Int -> Int
```

は、

```
add :: Int -> (Int -> Int)
```

と同じ意味である。このように、一つの引数を受け取り、残りの引数を受け取る関数を返す形をカリー化 (currying) と呼ぶ。

4.1 Python の return と Haskell の return

なお、Haskell にも return という名前の関数は存在する。しかし、それは Python の return 文とは意味が異なる。Haskell の return は、値を IO や他の文脈に持ち上げるための関数であり、関数か

ら値を返して処理を終了する構文ではない。そのため、Python の感覚で `return` を読むと混乱しやすい。

5 型宣言はコメントではない

Python でも以下のように型ヒントを書くことができる。また、現代の開発では必須であろう。

```
def add(x: int, y: int) -> int:  
    return x + y
```

ただし、Python の型ヒントは標準の実行時には基本的に強制されない。型検査をするには、mypy や pyright などの外部ツールを使うことが多い。

一方、Haskell の型はコンパイラによって検査される。

```
add :: Int -> Int -> Int  
add x y = x + y
```

この関数に文字列を渡そうとすると、コンパイル時にエラーになる。

Haskell では型宣言を書かなくても、コンパイラが多くの場合に型を推論する。

```
add x y = x + y
```

しかし、研究用コードでは明示的に型を書く方がよい。型宣言は単なる注釈ではなく、関数がどのような概念を扱うかを示す仕様書になるからである。

6 リスト処理：for 文から高階関数 (higher-order function) へ

Python では、リストを処理するときに for 文をよく使う。

```
xs = [1, 2, 3, 4]
ys = []

for x in xs:
    ys.append(x * x)

print(ys)
```

一方、Haskell では、リストを更新するのではなく、新しいリストを作る。

```
xs :: [Int]
xs = [1, 2, 3, 4]

ys :: [Int]
ys = map square xs
where
    square x = x * x
```

または、ラムダ式 (Lambda expression) を使って次のように書ける。

```
ys :: [Int]
ys = map (\x -> x * x) [1, 2, 3, 4]
```

Python のリスト内包表記に近い書き方もある。

```
ys = [x * x for x in xs]
```

Haskell では次のように書く。

```
ys :: [Int]
ys = [x * x | x <- xs]
```

条件を付ける場合、Python では次のように書く。

```
ys = [x * x for x in xs if x % 2 == 0]
```

Haskell では次のようになる。

```
ys :: [Int]
ys = [x * x | x <- xs, even x]
```

これは、

$$y_s = [x^2 | x \in x_s, \text{even } x]$$

のように理解すれば良い。

7 map, filter, fold

Pythonist にとって、Haskell のリスト処理を理解する鍵は、map, filter, fold である。

7.1 map

Python の

```
list(map(lambda x: x * x, [1, 2, 3]))
```

に対応する Haskell は、

```
map (\x -> x * x) [1, 2, 3]
```

である。

7.2 filter

Python の

```
[x for x in xs if x > 0]
```

に対応する Haskell は、

```
filter (> 0) xs
```

である。

ここで (> 0) は、値を一つ受け取って「0 より大きいかな」を返す関数である。

7.3 fold

Python では合計を次のように書ける。

```
total = 0
for x in xs:
    total += x
```

Haskell では、畳み込みとして書く。

```
total :: Int
total = foldr (+) 0 xs
```

Haskell では、畳み込みの代表的な関数として foldr と foldl がある。本ノートでは、まず基本形として foldr を用いる。実務上、大きなリストを左から畳み込む場合には、遅延評価によるメモリ使用を避けるために foldl' を使うことがある。

また、数値リストの合計なら単に

```
sum xs
```

でよい。

重要なのは、Haskell では「リストを順に更新していく」というより、「リストから値を構成する」と考える点である。

8 if 式

Python の if は文として使うことが多い。

```
if x > 0:
    y = "positive"
else:
    y = "non-positive"
```

Haskell の if は式である。つまり、値を返す。

```
signText :: Int -> String
signText x =
    if x > 0
    then "positive"
    else "non-positive"
```

Haskell では else を省略できない。なぜなら、if は式であり、必ず何らかの値を返さなければならないからである。

これは Python の三項演算子に近い。

```
y = "positive" if x > 0 else "non-positive"
```

Haskell では、条件分岐も値を作る式として扱う。

9 パターンマッチ

Haskell の特徴的な機能の一つがパターンマッチである。

例えば、リストの先頭を取り出す関数を考える。Python では次のように書ける。

```
def describe(xs):
    if len(xs) == 0:
        return "empty"
    else:
        return "non-empty"
```

Haskell では、リストの形で場合分けできる。

```
describe :: [a] -> String
describe []      = "empty"
describe (_,_)  = "non-empty"
```

ここで

```
[]
```

は空リストを表すパターンである。

```
(_:_)
```

は、先頭要素と残りのリストからなる非空リストを表すパターンである。`_` は、その値を使わないという意味である。

もう少し具体的に、先頭要素を返す関数を書くと次のようになる。

```
firstOrZero :: [Int] -> Int
firstOrZero []      = 0
firstOrZero (x:_) = x
```

パターンマッチは、Haskell における場合分けの中心的な道具である。Python の `if` や `match` に似ているが、Haskell では型と組み合わせることで非常に強力になる。

10 代数的データ型：class とは別のデータ設計

Python では、独自のデータ構造を作るときに class を使うことが多い。

```
class Payoff:
    def __init__(self, p1, p2):
        self.p1 = p1
        self.p2 = p2
```

Haskell では、データの形を data で定義する。

```
data Payoff = Payoff
  { payoffP1 :: Double
  , payoffP2 :: Double
  }
deriving (Eq, Show)
```

これは、Payoff という型を定義し、その値を作るコンストラクタも Payoff と名付けている。

Python の dataclass に近いものとして考えるとよい。

```
from dataclasses import dataclass

@dataclass
class Payoff:
    p1: float
    p2: float
```

ただし、Haskell の data は単なるレコード定義にとどまらない。複数の形を持つデータを定義できる。

```
data Result
  = Success Double
  | Failure String
deriving (Eq, Show)
```

これは、Result 型の値が

- Success Double
- Failure String

のどちらかであることを表す。

Python であれば、成功時は値を返し、失敗時は例外を投げることが多い。Haskell では、このような「成功か失敗か」を型で表現することができる。

11 Maybe : 存在しないかもしれない値

Python では、値が存在しない場合に `None` を使う。

```
def find_positive(xs):  
    for x in xs:  
        if x > 0:  
            return x  
    return None
```

Haskell では、存在しないかもしれない値を `Maybe` で表す。

```
findPositive :: [Int] -> Maybe Int  
findPositive [] = Nothing  
findPositive (x:xs)  
    | x > 0      = Just x  
    | otherwise = findPositive xs
```

`Maybe Int` は、「`Int` があるかもしれないし、ないかもしれない」という型である。

- `Just x` は値があることを表す。
- `Nothing` は値がないことを表す。

Python の `None` と違う重要な点は、Haskell では型に `Maybe` が現れることである。つまり、関数の型を見るだけで、「この関数は失敗する可能性がある」ことが分かる。

12 Either : 失敗理由を持つ結果

Maybe は失敗理由を持たない。失敗理由も返したい場合は Either を使う。

Python では、例えば次のように例外を投げるかもしれない。

```
def reciprocal(x):
    if x == 0:
        raise ValueError("division by zero")
    return 1 / x
```

Haskell では、失敗を値として返せる。

```
reciprocal :: Double -> Either String Double
reciprocal x
  | x == 0    = Left "division by zero"
  | otherwise = Right (1 / x)
```

Either String Double は、「失敗したら String、成功したら Double」という型である。

Haskell では慣習的に、

- Left err を失敗
- Right value を成功

として使うことが多い。

このように、Haskell では例外的な状況を型に現す設計がしやすい。研究用コードでは、この性質は重要である。入力ファイルのパース失敗、行列サイズの不一致、均衡計算の失敗などを、型として明示できるからである。

13 型を読むということ

Haskell では、関数の型を見るだけで、その関数が何をする可能性があるかをかなり推測できる。
例えば、

```
parseGameText :: String -> Either String Game
```

という型を見れば、この関数は文字列を受け取り、成功すれば `Game` を返し、失敗すればエラーメッセージとして `String` を返すことが分かる。

また、

```
readGameFile :: FilePath -> IO (Either String Game)
```

という型であれば、この関数はファイル入出力を行い、その結果として成功または失敗を返すことが分かる。

Python では、このような情報は関数名、ドキュメント、例外仕様、慣習に分散しやすい。Haskell では、それらの一部が型に現れる。したがって、Haskell では「型を読む」ことが、そのままプログラムの設計意図を読むことにつながる。

14 レコード構文

Haskell のレコード構文は、Python の `dataclass` に近い感覚で使える。

```
data Cell = Cell
  { cellRowStrategy :: RowStrategy
  , cellColStrategy :: ColStrategy
  , cellPayoff      :: Payoff
  }
deriving (Eq, Show)
```

この定義によって、次のようなアクセサ関数が自動的に作られる。

```
cellRowStrategy :: Cell -> RowStrategy
cellColStrategy :: Cell -> ColStrategy
cellPayoff      :: Cell -> Payoff
```

値を作るときは、次のように書ける。

```
cell :: Cell
cell = Cell
  { cellRowStrategy = RowStrategy "U"
  , cellColStrategy = ColStrategy "L"
  , cellPayoff      = Payoff 3.0 2.0
  }
```

Python の `dataclass` であれば、次のような雰囲気である。

```
cell = Cell(
    row_strategy=RowStrategy("U"),
    col_strategy=ColStrategy("L"),
    payoff=Payoff(3.0, 2.0),
)
```

Haskell のレコードは、データ構造を明示するために非常に便利である。特に、ゲーム理論のように「戦略」「利得」「セル」「ゲーム」といった概念をコードに写す場合に有効である。

15 newtype : 意味の違いを型で表す

Python では、行プレイヤーの戦略名も列プレイヤーの戦略名も、単なる文字列として扱うことが多い。

```
row_strategy = "U"
col_strategy = "L"
```

しかし、これでは両者を取り違えても実行時まで気づきにくい。

Haskell では、`newtype` を使って、実行時表現は同じでも型として区別できる。

```
newtype RowStrategy = RowStrategy { unRowStrategy :: String }
    deriving (Eq, Ord, Show)

newtype ColStrategy = ColStrategy { unColStrategy :: String }
    deriving (Eq, Ord, Show)
```

これにより、`RowStrategy` と `ColStrategy` はどちらも中身は `String` だが、型としては別物になる。

これは研究コードではかなり重要である。数理モデルでは、同じような表現を持つが意味の異なる対象が頻繁に現れる。Haskell の `newtype` は、その意味の違いを型レベルで表すための軽量な道具である。

16 純粋関数と副作用

Haskell を理解する上で重要なのが、純粋関数と副作用の分離である。

Python では、関数の中で表示、ファイル読み込み、乱数生成、変数更新などを自然に行える。

```
def greet(name):  
    print("Hello, " + name)
```

Haskell では、画面表示は副作用なので、型に IO が現れる。

```
greet :: String -> IO ()  
greet name = putStrLn ("Hello, " ++ name)
```

IO () は、「IO を行い、意味のある値は返さない」という型である。

一方、純粋な文字列生成は IO を持たない。

```
makeGreeting :: String -> String  
makeGreeting name = "Hello, " ++ name
```

そして、それを表示する部分だけが IO になる。

```
main :: IO ()  
main = putStrLn (makeGreeting "Alice")
```

この分離は、Haskell の大きな利点である。計算そのものと、表示・ファイル読み書き・外界とのやり取りを分けられるからである。

研究コードでは、これは特に有効である。解析ロジックは純粋関数として書き、ファイル入出力や表示は外側に置くことで、テストしやすく、再利用しやすいコードになる。

17 main :: IO () の読み方

Haskell の実行可能プログラムには、通常次のような定義がある。

```
main :: IO ()
main = do
  putStrLn "Hello, Haskell"
```

Pythonist にとっては、main は次のようなものに近い。

```
def main():
    print("Hello, Haskell")

if __name__ == "__main__":
    main()
```

ただし、Haskell では main の型が明示される。

```
main :: IO ()
```

これは「このプログラムの入口は IO アクションである」という意味である。

do 記法は、複数の IO アクションを順番に並べるための構文である。

```
main :: IO ()
main = do
  putStrLn "Input your name:"
  name <- getLine
  putStrLn ("Hello, " ++ name)
```

ここで

```
name <- getLine
```

は、getLine という IO アクションから結果を取り出し、それを name に束縛している。通常の純粋な let 束縛とは意味が異なる。

注意すべきなのは、IO String は String そのものではないという点である。例えば、getLine の型は次のようになる。

```
getLine :: IO String
```

これは「文字列」ではなく、「実行すると文字列を得る IO アクション」である。したがって、通常の純粋関数にそのまま渡すことはできない。do 記法の中で

```
name <- getLine
```

と書くことで、IO アクションの結果として得られた String を name に束縛できる。

18 let と where

Haskell では、補助的な値や関数を定義するために `let` や `where` を使う。

18.1 let

```
circleArea :: Double -> Double
circleArea r =
  let piValue = pi
      r2 = r * r
  in piValue * r2
```

`let` は、局所的な定義を作って、その後の式で使う。

18.2 where

```
circleArea :: Double -> Double
circleArea r = piValue * r2
  where
    piValue = pi
    r2 = r * r
```

`where` は、式の後ろに補助定義を書く。

どちらもよく使われる。小さな一時変数なら `let`、関数全体を支える補助定義なら `where` が読みやすいことが多い。

Pythonist にとっては、関数内のローカル変数定義に近いが、Haskell ではこれらも式の一部として扱われる。

19 インデントは構文である

Python と同様に、Haskell でもインデントは重要である。ただし、Python のインデントとは規則が少し違う。

例えば、次のように `where` の下の定義は揃える必要がある。

```
areaAndCircumference :: Double -> (Double, Double)
areaAndCircumference r = (area, circumference)
  where
    area = pi * r * r
    circumference = 2 * pi * r
```

Haskell では、同じブロックに属する定義の開始位置を揃えることが重要である。

Pythonist はインデントに慣れているため、この点では Haskell に入りやすい。ただし、Haskell では `where`, `let`, `do`, `case` など、複数の構文でレイアウト規則が効くため、最初は小さな例を写経して慣れるのがよい。

20 遅延評価：必要になるまで計算しない

Haskell は遅延評価を採用している。これは、値が必要になるまで計算を遅らせるという意味である。

例えば、Haskell では無限リストを自然に書ける。

```
numbers :: [Integer]
numbers = [1..]
```

これは 1 から始まる無限リストである。しかし、定義した瞬間に無限に計算するわけではない。必要になった分だけ評価される。

例えば、

```
firstTen :: [Integer]
firstTen = take 10 numbers
```

とすれば、最初の 10 個だけが必要になる。

Python では、これに近いものとしてジェネレータがある。

```
def numbers():
    n = 1
    while True:
        yield n
        n += 1
```

Haskell では、こうした遅延的な構成が言語全体の基本に組み込まれている。これは強力だが、性能やメモリ使用量を考える場合には注意も必要である。

21 型クラス：Python の duck typing との距離感

Python では、あるオブジェクトが特定のメソッドを持っていれば、その型に関係なく使えることが多い。これは duck typing と呼ばれる。

Haskell には、これに少し似た役割を持つ型クラスがある。ただし、Haskell の型クラスは静的に検査される。

例えば、等値比較できる型は Eq 型クラスのインスタンスである。

```
(==) :: Eq a => a -> a -> Bool
```

これは、「a が Eq のインスタンスであれば、a 同士を比較して Bool を返せる」という意味である。表示可能な型は Show 型クラスのインスタンスである。

```
show :: Show a => a -> String
```

自分で定義した型に対して、自動的に Eq や Show を導出することもできる。

```
data Payoff = Payoff Double Double
deriving (Eq, Show)
```

Python の duck typing は実行時に柔軟である。Haskell の型クラスは、柔軟性を持ちつつも、コンパイル時に整合性を確認できる仕組みである。

22 モジュールと import

Python では、ファイルやパッケージを import する。

```
from game_theory.types import Game
from game_theory.parser import parse_game
```

Haskell でも、モジュールを import する。

```
import GameTheory.NormalForm.Types
import GameTheory.NormalForm.Parser
```

Haskell では、モジュール名とファイル位置を対応させるのが基本である。

モジュール名	ファイル位置
GameTheory.NormalForm.Types	src/GameTheory/NormalForm/Types.hs
GameTheory.NormalForm.Parser	src/GameTheory/NormalForm/Parser.hs

Pythonist が注意すべきなのは、Haskell では src を import 名に含めないことである。

```
import GameTheory.NormalForm.Types
```

であり、

```
import src.GameTheory.NormalForm.Types
```

ではない。

23 Pythonist が最初に作るとよい Haskell プロジェクト

Pythonist が Haskell を試すとき、単一ファイルから始めてもよい。しかし、研究用コードや教材コードとして育てるなら、最初から小さな Cabal プロジェクトにする方がよい。

例えば、次のような構成である。

```
my-haskell-notes/  
  my-haskell-notes.cabal  
  src/  
    Examples/  
      Basics.hs  
      Lists.hs  
      Types.hs  
  app/  
    Main.hs  
  test/  
    Spec.hs
```

src/ には再利用したいコードを置く。app/ には実行入口を置く。test/ にはテストを書く。

Python の感覚で言えば、src/ がライブラリ本体、app/Main.hs が小さな CLI スクリプトに近い。

24 Cabal の最小例

最小限の Cabal ファイルは次のようになる。

```
cabal-version:      3.0
name:               my-haskell-notes
version:            0.1.0.0
build-type:         Simple

library
  hs-source-dirs:   src
  exposed-modules:
    Examples.Basics
    , Examples.Lists
    , Examples.Types
  build-depends:
    base >=4.14 && <5
  default-language: Haskell2010
  ghc-options:      -Wall

executable my-haskell-notes
  hs-source-dirs:   app
  main-is:          Main.hs
  build-depends:
    base >=4.14 && <5
    , my-haskell-notes
  default-language: Haskell2010
  ghc-options:      -Wall
```

この構成では、app/Main.hs から src/ 側のモジュールを import できる。

```
module Main where

import Examples.Basics

main :: IO ()
main = putStrLn (makeGreeting "Pythonist")
```

src/Examples/Basics.hs は次のように書ける。

```
module Examples.Basics
  ( makeGreeting
  ) where

makeGreeting :: String -> String
makeGreeting name = "Hello, " ++ name
```

ビルドと実行は次のように行う。

```
cabal build
```

```
cabal run
```

25 Python から Haskell へ移るときの読み替え表

最後に、Pythonist 向けの読み替え表をまとめておく。

Python の感覚	Haskell の対応	注意点
変数代入	束縛	基本的に再代入しない
関数定義	関数定義	本体の式が返り値
型ヒント	型宣言	コンパイラが検査する
None	Maybe	型に失敗可能性が出る
例外	Either / IO 例外	失敗を値として扱える
dataclass	data / record	型による構造化が中心
文字列ラッパー	newtype	意味の違いを型で表す
for 文	map / filter / fold / 再帰	値の更新ではなく構成
list comprehension	list comprehension	Haskell にもある
module import	import	論理モジュール名を書く
script main	main :: IO ()	IO は型で明示される
duck typing	type class	静的に制約される
generator	lazy list	遅延評価が標準

26 Pythonist がつまずきやすい点

Pythonist が Haskell を学ぶとき、つまずきやすい点はかなりはっきりしている。

1. 変数を更新できないことに戸惑う。
2. for 文ではなく map, filter, fold や再帰で考える必要がある。
3. 型エラーが多く見える。
4. IO が通常の値と分離されていることに戸惑う。
5. Maybe や Either を面倒に感じる。
6. Cabal, GHC, GHCup, Stack などの道具の関係が分かりにくい。

しかし、これらは単なる障害ではない。むしろ、Python では暗黙に処理していたことを、Haskell では明示していると見ると理解しやすい。

例えば、None が返るかもしれない関数は、Python ではドキュメントや慣習に依存しがちである。Haskell では Maybe が型に現れる。

例外を投げるかもしれない処理も、Haskell では Either によって値として扱える。

ファイル読み込みや画面表示も、Haskell では IO として型に現れる。

つまり、Haskell は Python よりも面倒なのではなく、暗黙の前提をコード上に出す言語だと考えるとよい。

27 研究コードとしての Haskell

Python は、実験、データ処理、可視化、機械学習に非常に強い。そのため、研究用コードでは Python が第一選択になることが多い。

一方、Haskell は、次のような場面で強みを持つ。

- 数理モデルの構造を型で明示したい。
- 状態、遷移、結果、失敗を区別したい。
- 解析ロジックと IO を分けたい。
- 小さな DSL を設計したい。
- アルゴリズムの意味構造を明示したい。
- 実装を理論記述の補助として使いたい。

したがって、Python と Haskell は競合するというより、役割が異なる。

例えば、データの前処理、可視化、統計処理、機械学習の実験は Python が強い。一方、ゲーム理論、意思決定理論、形式的な状態遷移、パーサ、DSL、型による概念整理では Haskell が強みを持つ。

研究ワークフローとしては、次のような使い分けが考えられる。

作業	Python	Haskell
データ収集・整形	強い	可能だが主戦場ではない
可視化	非常に強い	外部連携が必要
数値計算	ライブラリが豊富	目的次第
理論モデルの記述	書けるが緩い	型で明示しやすい
パーサ・DSL	可能	非常に相性がよい
状態遷移モデル	可能	型で整理しやすい
実験スクリプト	強い	小規模なら可
概念構造の明示	弱くはない	強い

28 まとめ

本ノートでは、Pythonist の視点から Haskell の基本的な考え方を整理した。
要点は次の通りである。

1. Haskell では、変数を更新するより、値を構成する。
2. 関数は入力から出力を作る純粋な写像として考える。
3. 型宣言はコメントではなく、コンパイラが検査する仕様である。
4. Maybe や Either によって、失敗可能性を型に表せる。
5. data や newtype によって、概念の違いをコード上に明示できる。
6. IO によって、純粋な計算と副作用を分離できる。
7. map, filter, fold は、Python の for 文的な処理を関数型的に捉え直す入口である。
8. Haskell の module/import は、研究コードを資産化するための重要な仕組みである。

Python から Haskell へ移ると、最初は多くのことが窮屈に見える。しかし、その窮屈さの多くは、暗黙の前提を明示するための制約である。

Python は、素早く実験し、データを扱い、外部ライブラリを活用するために非常に優れている。Haskell は、概念構造を明示し、型によって不整合を減らし、理論的なモデルをコードとして表現するために優れている。

したがって、Pythonist にとって Haskell は、Python の代替というより、思考の別モードを与える言語である。特に、数理モデル、ゲーム理論、意思決定理論、形式的なアルゴリズム記述、DSL 設計に関心がある場合、Haskell は単なるプログラミング言語以上に、概念を整理するための道具として機能する。

付録 A Appendix : 最初に覚える Haskell コマンド

Haskell の学習を始めるとき、最低限覚えておくとよいコマンドをまとめる。

コマンド	意味
ghci	対話環境を起動する
ghci Main.hs	ファイルを読み込んで GHCi を起動する
:t expression	GHCi で式の型を確認する
:r	GHCi でファイルを再読み込みする
ghc Main.hs	単一ファイルをコンパイルする
runghc Main.hs	コンパイル済み実行ファイルを作らずに Haskell ファイルを実行する
cabal build	Cabal プロジェクトをビルドする
cabal run	Cabal プロジェクトの実行ファイルを実行する
cabal test	テストを実行する
ghcup list	GHCup で利用可能なツール一覧を見る

最初は、ghci で小さな式を試し、慣れてきたら Cabal プロジェクトを作るのがよい。Python で REPL や Jupyter Notebook を使って小さく試すのと同様に、Haskell でも GHCi を使って型や関数の挙動を確認すると理解しやすい。

付録 B Appendix : Pythonist 向けの最小 Haskell ファイル

最後に、Pythonist が最初に試すための最小ファイルを示す。

```
module Main where

square :: Int -> Int
square x = x * x

positives :: [Int] -> [Int]
positives xs = filter (> 0) xs

describe :: [a] -> String
describe []    = "empty"
describe (_:_) = "non-empty"

safeHead :: [a] -> Maybe a
safeHead []    = Nothing
safeHead (x:_) = Just x

main :: IO ()
main = do
  print (square 5)
  print (positives [-2, 0, 3, 4])
  putStrLn (describe [1, 2, 3])
  print (safeHead [10, 20, 30])
```

このファイルを Main.hs として保存し、

```
ghci Main.hs
```

または

```
runghc Main.hs
```

で試すとよい。

ここには、Haskell の基本要素が小さく含まれている。

- 型宣言
- 純粋関数
- filter
- パターンマッチ
- Maybe

- IO

Pythonist にとっては、この小さな例を何度も書き換えることが、Haskell の発想に慣れるための第一歩になる。